

EEET2588

Real-time Systems Engineering & Design

Lecture 2: Concurrent Processes

- Thread Attributes
- Message Queues and Message Passing

Dr Hung Pham

Course note acknowledgment: Original course notes developed by Roy Ferguson and Samuel Ippolito from RMIT Melbourne

Lecture 1 Review

- **Course Overview**

- Syllabus, References, Assessments

- **What is a Real-Time System**

- Importance of Time (deadlines)
- Hard, Soft, Firm and Weakly hard Systems

- **Overview of RT System Design**

- An extension of standard System Design

- **Different OS Overview**

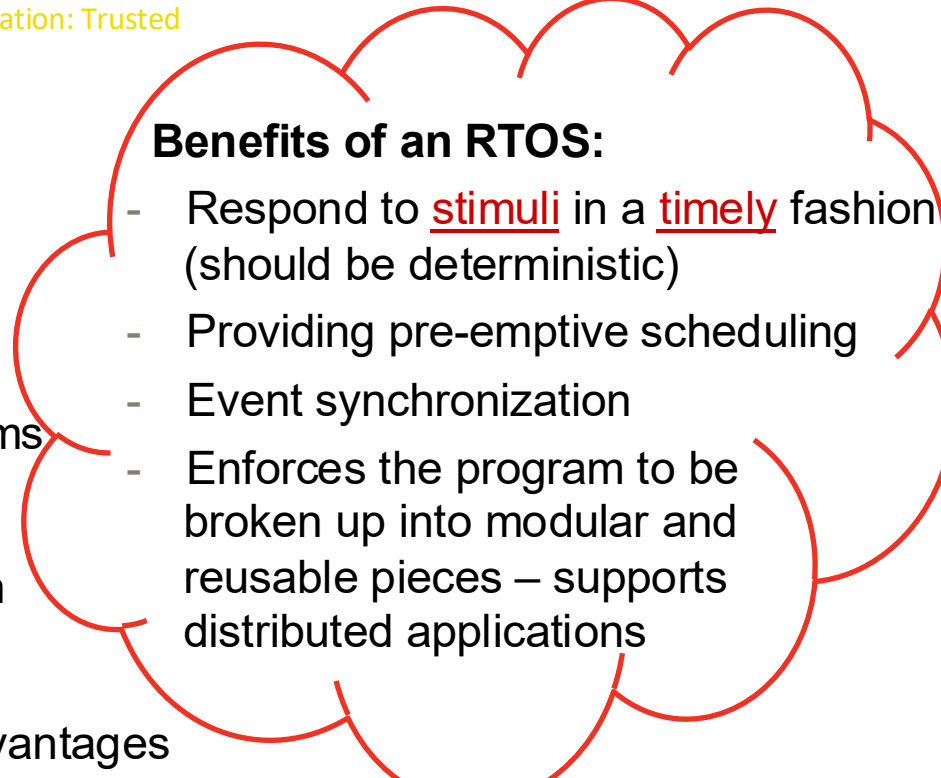
- Purpose, Types, Advantages and Disadvantages

- **RT Extensions**

- RT Systems build on conventional OS

- **Features provided by an RTOS**

- User has control of OS policies
- Highly responsive
 - Fast task/context switching
 - Fast interrupt response
- Limited security/protection
- Executes in precisely the same amount of time every time
- Handles multiple competing events
- Events can occur simultaneously
- Provides synchronization, messaging, etc.

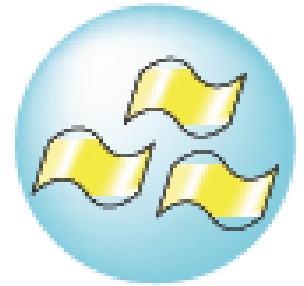


Benefits of an RTOS:

- Respond to stimuli in a timely fashion (should be deterministic)
- Providing pre-emptive scheduling
- Event synchronization
- Enforces the program to be broken up into modular and reusable pieces – supports distributed applications

Lecture 2 Overview

- Processes and Threads
 - Returning a value from a Thread
 - Joined and Detached Threads
 - Changing Thread Attributes (ie. scheduling parameters)
 - **Errno** (checking that functions successfully execute correctly)
- POSIX Message Queues
- QNX Message Passing
 - Channel Connect
 - Named Connection



A process with three active threads [1]

Note: Information on the QNX functions can be found by typing name in to the search box in the following link: <http://www.qnx.com/developers/docs/7.1/>

Laboratory program

This week: Intro to QNX – Exercise 1

Next week: You will be working on Threads and Message Queues – Ex2

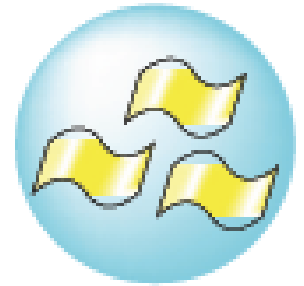


Week	Laboratories
1	No Labs in week 1
2	Ex1 – Intro to QNX and Momentics
3	Ex2 – Threads & Queues
4	Ex3 – Shared memory and Mutual Exclusion
5	Ex4 – State Machines with threads
6	Ex5 – Message Passing between process and nodes
8	Project Work
9	Project Work
10	Project Work
11	Project Work
12	No Lab - Project Assessment Week

Remember to submit your Lab 1 report by Friday week 3

Processes and Tasks

- Process, task and threads.
 - The term ‘Process’ and ‘Task’ are used interchangeably.
- What is a **PROCESS** (TASK) ?
 - **“A Program in Execution”**
 - A Sequence of code executing
 - Includes program code as it resides in memory plus current activity
 - e.g. PC and register values, stack and heap contents, open files etc.
 - A process itself is a Thread, however it can hold multiple other threads
 - When a process has more than one thread it is said to have concurrently executing code.
- In QNX, there are 21 possible states a process/thread can be in
 - **Essential Process States**
 - Running executing - Only one such process if a single CPU
 - Ready to run if a CPU were free
 - Blocked waiting for an event
 - e.g. I/O completion
 - Stopped/Dead



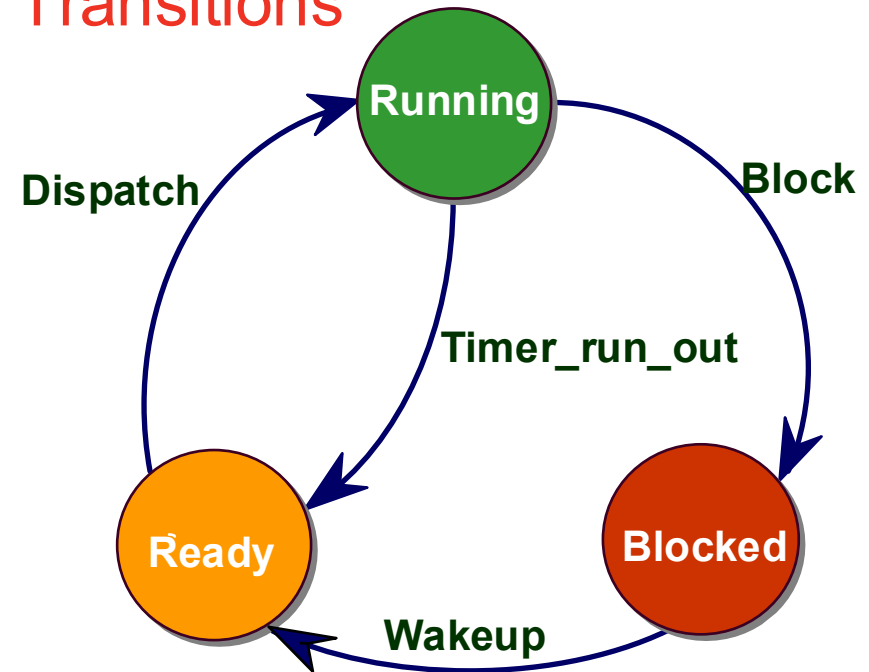
A process with three active threads [1]



Three threads in two different processes [1]

Process Control Block and State Transitions

- The scheduler (kernel) holds information about all executing processors (and their threads)
- The OS uses the information to control the state of the active processes e.g.
 - Current State of process
 - Process ID
 - Program Counter
 - CPU register values
 - e.g. Stack Pointer
 - CPU scheduling information
 - e.g. process priority
 - Memory Management information
 - limit register values or page table pointers
 - Accounting information
 - CPU time, etc.
 - I/O status information
 - e.g. open files, I/O devices used



192.168.1.106 - PuTTY

```
# pidin
```

pid	tid	name	prio	STATE	Blocked
1	1	boot/procnto-instr	0f	READY	
1	2	boot/procnto-instr	255r	RECEIVE	1
1	3	boot/procnto-instr	255r	RECEIVE	1
1	4	boot/procnto-instr	10r	RECEIVE	1
1	5	boot/procnto-instr	255r	RECEIVE	1
1	6	boot/procnto-instr	10r	RECEIVE	1
1	7	boot/procnto-instr	10r	RECEIVE	1
1	8	boot/procnto-instr	10r	RECEIVE	1
1	9	boot/procnto-instr	10r	RECEIVE	1
1	10	boot/procnto-instr	10r	RUNNING	
1	11	boot/procnto-instr	10r	RECEIVE	1
1	12	boot/procnto-instr	10r	RECEIVE	1
2	1	proc/boot/slogger	21r	RECEIVE	1
3	1	proc/boot/pipe	10r	SIGWAITINFO	

QNX Kernel States

- When a thread is blocked, it consumes no CPU time (Running is the exception)
- list is available in `<sys/neutrino.h>`

If the state is:	The thread is:
CONDVAR	Waiting for a condition variable to be signaled.
DEAD	Dead. Kernel is waiting to release the thread's resources.
INTR	Waiting for an interrupt.
JOIN	Waiting for the completion of another thread.
MUTEX	Waiting to acquire a mutex.
NANOSLEEP	Sleeping for a period of time.
NET_REPLY	Waiting for a reply to be delivered across the network.
NET_SEND	Waiting for a pulse or message to be delivered across the network.
READY	Not running on a CPU, but is ready to run (one or more higher or equal priority threads are running).
RECEIVE	Waiting for a client to send a message.
REPLY	Waiting for a server to reply to a message.
RUNNING	Actively running on a CPU.
SEM	Waiting to acquire a semaphore.
SEND	Waiting for a server to receive a message.
SIGSUSPEND	Waiting for a signal.
SIGWAITINFO	Waiting for a signal.
STACK	Waiting for more stack to be allocated.
STOPPED	Suspended (SIGSTOP signal).
WAITCTX	waiting for a register context (usually floating point) to become available (only on SMP systems).
WAITPAGE	Waiting for process manager to resolve a fault on a page.
WAITTHREAD	Waiting for a thread to be created.

Concurrent Behaviour

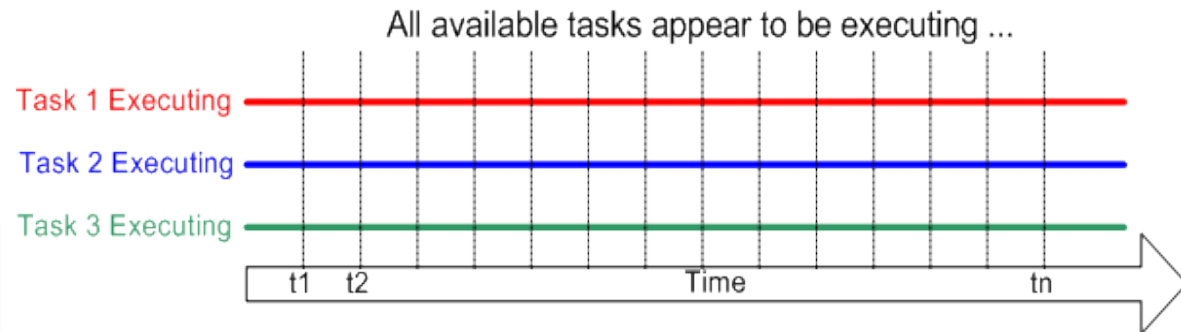
- All statements start “simultaneously” and execute concurrently
- Process may **block** until all threads have completed executing (ie: **Join**). Alternatively processes may have **detached** threads, or **fork** to other processes.
- When analyzing the behaviour of concurrent processes/threads
 - Make no assumptions about the relative speeds of the concurrent processes
 - or when a process switches might occur (even on a single CPU node)



```
int main()
{
    int val = 5;
    funct1(&val);
    sleep(6);
    ...
}
```



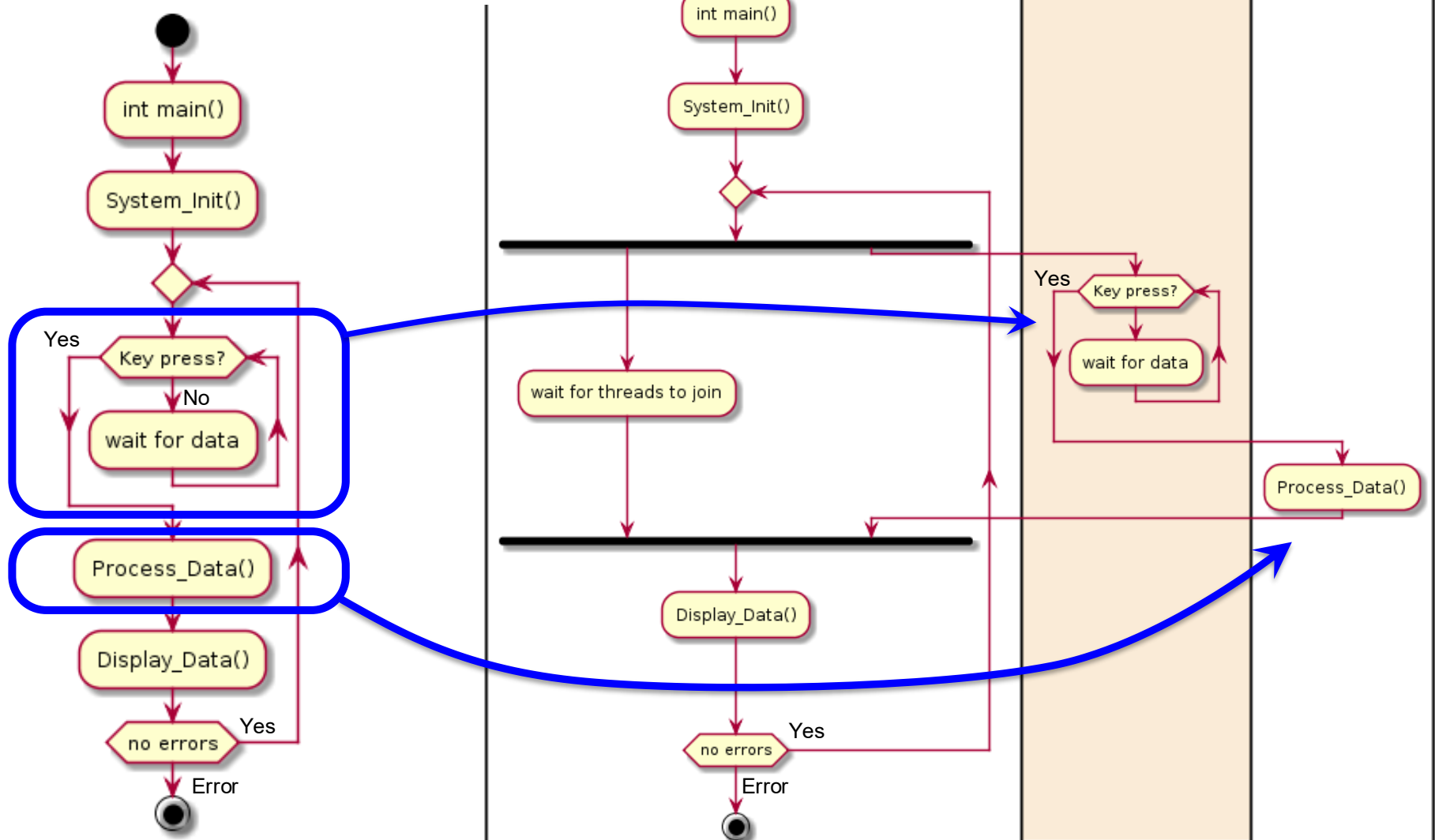
```
int main()
{
    int dat = 5;
    print(&dat);
    message(6);
    ...
}
```



Concurrent threads executing on a single core CPU [2]

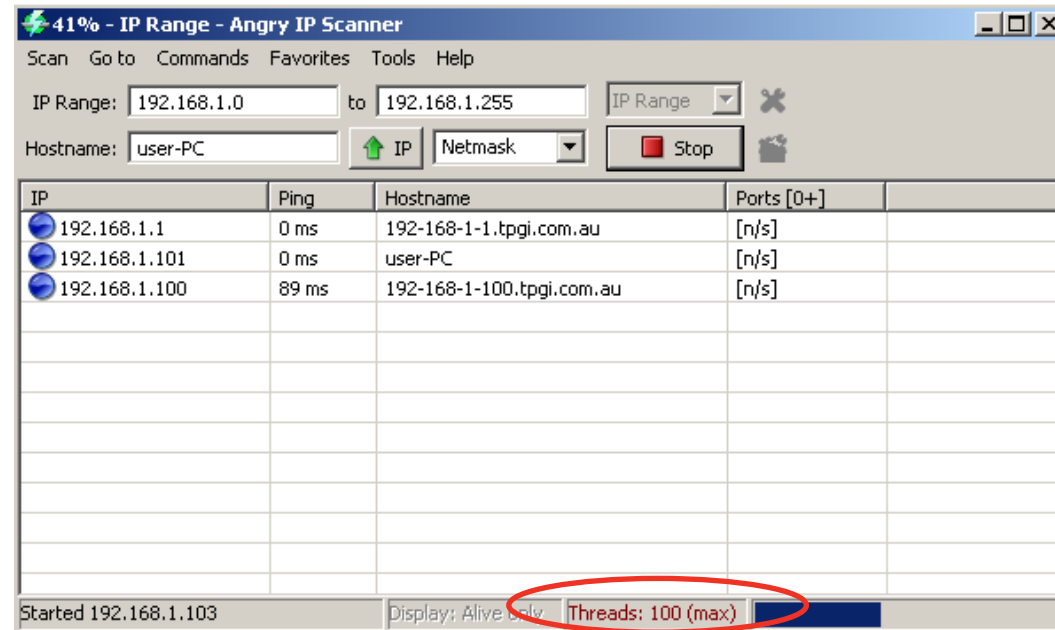
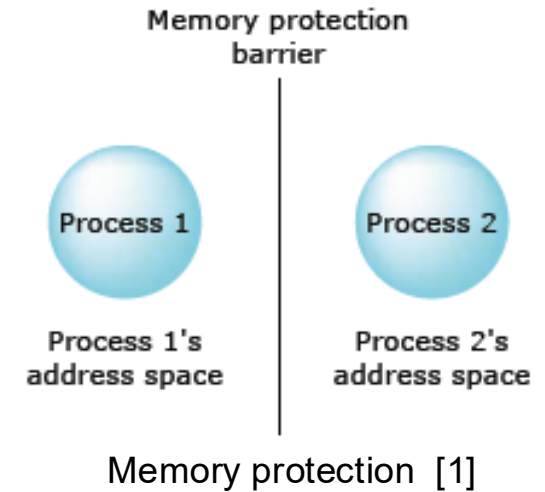
Concurrent Behaviour within a Single Processes

- Lets take a simple program and split it up into separate threads



Process and Threads

- Threads also known as lightweight processes
 - Threads **Share code and global data** with their parent process but have their own register values and stack
 - e.g a word processor could have threads for background spell checking and printing
- Processes can be a single thread process
- So why not just have one process with a zillion threads?
- The advantages of breaking things up into multiple processes are many:
 - Decoupling and modularity**
 - Maintainability**
 - Reliability**

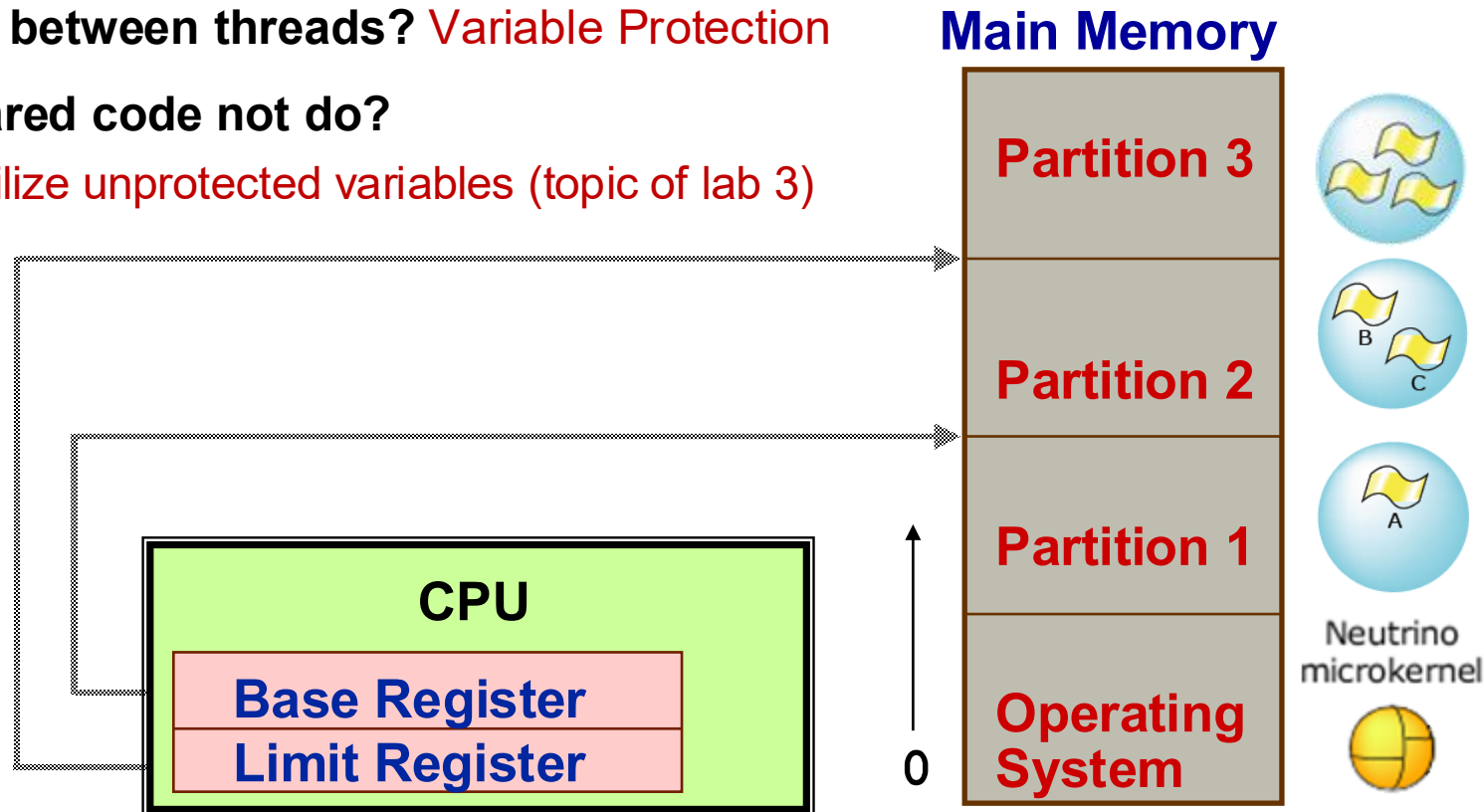


Other references on processes /threads

- *“Operating Systems”*, H.M. Deitel etc, Addison-Wesley
- *“Operating System Concepts”*, Peterson/Silberschatz/Galvin, Addison-Wesley
- *“Modern Operating Systems”*, A.S. Tanenbaum, Prentice-Hall

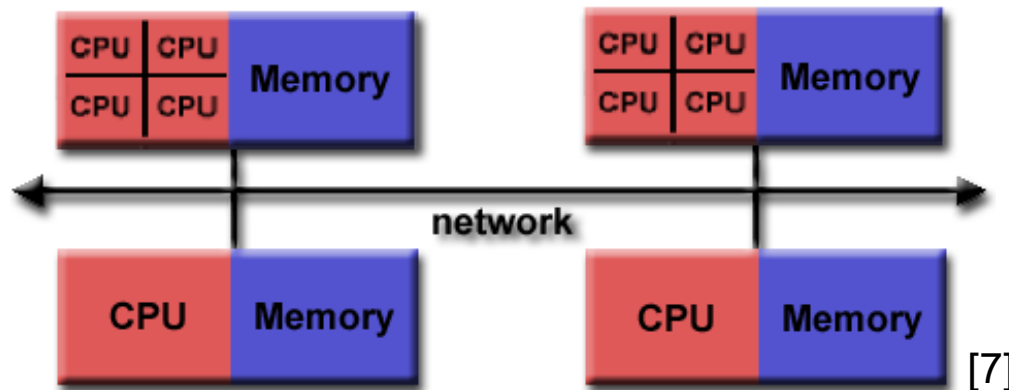
Resource Pooling

- **Processes do not share address space**
- **Threads within a process share everything within a process's address space**
 - Each thread can still have private data (scope is maintained)
 - Each thread has a stack for its own
- **What is required to safely share code/resources with a process between threads? Variable Protection**
- **What must shared code not do?**
Not share or utilize unprotected variables (topic of lab 3)

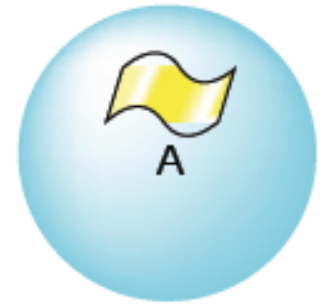


Concurrent Processes

- **Quasi-concurrent (pseudo-concurrent)**
 - Single Core CPU (serial processing with time slicing or cooperative scheduling)
- **Truly Concurrent**
 - Multiple CPUs or Cores including distributed computing, parallel processing
- **Two major categories**
 - Asynchronous Concurrent Processes
 - Require occasional synchronization and cooperation (loosely coupled)
 - Typical of operating systems, distributed processes, and real-time systems
 - [Example: Server-client type applications, webserver, etc.](#)
 - Synchronous Concurrent Processes
 - Tightly coupled – typical of parallel processing
 - [Example: GPU based computation, CFD calculations, etc.](#)



Using Threads within a Process



- We are use to writing processes with 1 thread inside them
- This example passes some information to a function
 - Sequential operation
 - Nothing runs in parallel – even on a multi-core CPU

```
#include <stdlib.h>

void Myfunction(void *data)
{
    int a = *((int*)(data));    // pointer casting
    printf("Function Started, sleep for %is\n",a);
    sleep(a);
    printf("Function Finished\n");
}

int main(int argc, char *argv[])
{
    int val = 5;
    Myfunction(&val);
    sleep(6);                  // block for 6 seconds
    printf("Main terminated\n");
    return EXIT_SUCCESS;
}
```

See [Beej's Guide to C](#) or the [Teach Yourself C++ in 21 Days](#) if your type casting and pointer operations are a little rusty!!

Note by using:

```
int *a = (int*)(data);
You can change the value
of val in main().
```

What is the total time this program will run for?

Getting Useful Process and Thread Information

- There are various functions that will return runtime information about a thread which can be useful to a programmer

- *Get the name of the current host*

```
int gethostname(char *name, size_t namelen); #include <unistd.h>
```

- Get the priority of a given process

```
int getprio( pid_t pid ); #include <sched.h> (QNX 4)
```

- Get the process ID

```
pid_t getpid( void ); #include <unistd.h>
```

- Get the thread ID

```
int gettid( void ); #include <process.h> (QNX)
```

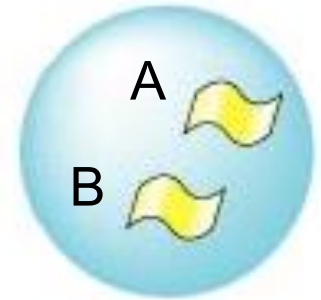
- *Get the parent process ID*

```
pid_t getppid( void ); #include <sys/types.h> #include <unistd.h>
```

```
void Thread_info() {
    char hostnm[BUF_SIZE] = {}; // #define BUF_SIZE 256
    memset(hostnm, '\0', BUF_SIZE);
    hostnm[BUF_SIZE - 1] = '\n';
    gethostname(hostnm, sizeof(hostnm));
    printf(" --> Machine hostname is: '%s'\n", hostnm);
    printf(" --> The parent process ID is  %d\n", getppid() );
    printf(" --> The process ID is %d\n",      getpid() );
    printf(" --> The thread ID is %d\n\n",     gettid() );
}
```


Multi-Thread Process Example

- A thread routine is called much like a function routine, however the thread is invoked using `pthread_create()`
- Example of a process that starts a thread
 - **Thread A** is the parent thread (runs for 6 seconds)
 - **Thread B** is created at runtime (only runs for 5 seconds)



The thread code function is not called directly but passed as a parameter to `pthread_create()`. The single void parameter may be used to pass parameters.

```
#include <stdlib.h>
#include <pthread.h>

void *thread_ex(void *data)
{
    int a = *((int*)(data));
    printf("Thread Started, sleep for %is\n",a);
    sleep(a);
    printf("Thread Finished\n");
}

int main(int argc, char *argv[])
{
    int val = 5;
    pthread_create(NULL, NULL, thread_ex, &val);
    sleep(6); // block for 6 seconds
    printf("Main terminated\n");
    return EXIT_SUCCESS;
}
```

```
#include <pthread.h>

int pthread_create(
    pthread_t* thread,
    const pthread_attr_t* attr,
    void* (*start_routine)(void* ),
    void* arg );
```

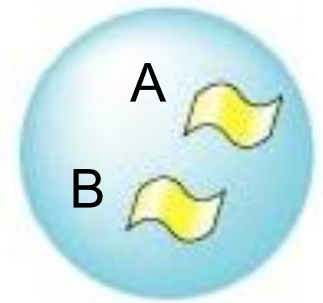
Arguments point to objects holding:

- **Thread ID** of the new thread
- Thread **attributes** (priority, etc.)
- **Location of thread routine**
- **Arguments passed to the thread**

In this example only a pointer to the thread routine and argument is passed.

Joining Threads and Returning a Value

- `pthread_join()` blocks the calling thread until the target thread terminates, unless thread has already terminated.



```
#include <stdlib.h>
#include <pthread.h>

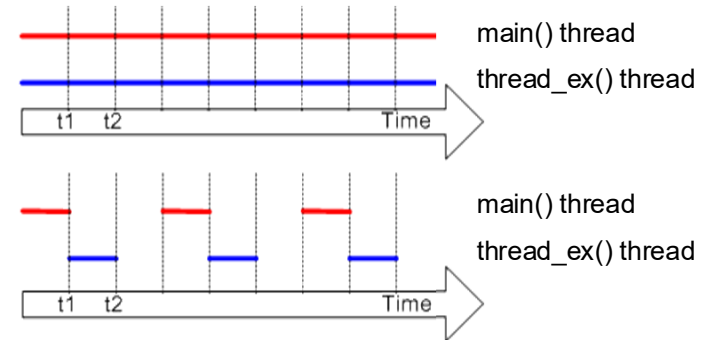
void *thread_ex(void *data)
{
    int a = *((int*)(data));
    printf("Thread Started, sleep for %is\n",a);
    sleep(a);
    printf("Thread Finished\n");
    return (int)152; // return a value
}

int main(int argc, char *argv[])
{
    pthread_t th1;
    int val = 5;
    void *retval;

    pthread_create(&th1, NULL, thread_ex, &val);

    sleep(2); // waste some time
    pthread_join(th1, &retval); // block here

    printf("Thread returned %i\n", retval);
    printf("Main terminated\n");
    return EXIT_SUCCESS;
}
```



```
#include <pthread.h>
int pthread_join( pthread_t thread,
                  void** value_ptr );
```

Arguments point to objects holding:

- **Thread ID** of target thread whose termination you're waiting for.
- Location where the function can store the value passed to `pthread_exit()` by the target thread. (ie. The *return* value)

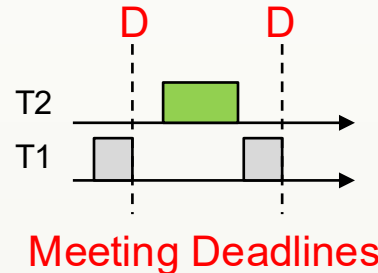
Joining In a Timely Fashion – Did I miss my deadline?

- The non-POSIX `pthread_timedjoin()` function is similar to `pthread_join()`, except that an **error** code is returned if the join doesn't occur before a given time.

```
#include <errno.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#include <string.h> //must add for QNX7

void *thread_ex(void *not_used) {
    printf("Thread Started\n");
    sleep(4);
    printf("Thread Finished\n");
}

void main() {
    pthread_t th1;
    struct timespec t_deadline;
    clock_gettime(CLOCK_REALTIME, &t_deadline);
    printf("Time is = %ld\n", t_deadline.tv_sec);
    t_deadline.tv_sec += 5; // Add 5 seconds (deadline)
    pthread_create(&th1, NULL, thread_ex, NULL);
    int retcode = pthread_timedjoin(th1, NULL, &t_deadline);
    if(retcode == EOK)
        printf("Successfully completed in time\n");
    else
        printf("Error %i: %s\n", retcode, strerror(retcode));
    printf("Main terminated\n");
}
```



errno Global error variable
`strerror()` – error to str

```
struct timespec {
    time_t    tv_sec;
    long      tv_nsec; }
```

```
int pthread_timedjoin(
    pthread_t thread,
    void** value_ptr,
    const struct timespec* abstime );
```

- Thread ID** that we're waiting for.
- Location for the threads **return** value
- A pointer to a **timespec** structure that specifies the maximum time to wait for the join. Must be: **CLOCK_REALTIME**

Detecting Error During Execution

- Many of the POSIX and QNX functions return error values if they are unable to perform a task. There are several error function libraries available.
- An important one is the `#include <errno.h>` library, which uses a global variable, `extern int errno;` which is set by supported functions.
- The following program makes an illegal call to the `write()` function, then prints the value held in `errno`

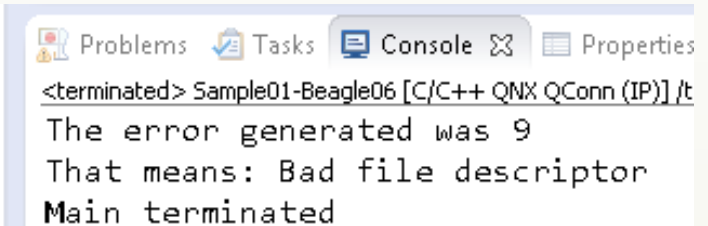
```
#include <stdio.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>

void main (void)
{
    int errvalue;
    errno = EOK; // Initialize the errno global value

    // do something that will cause an error
    write( -1, "hello, world\n", strlen( "hello, world\n" ) );
    errvalue = errno; // save the error (even if it didn't happen)

    // use the error value to display the appropriate error messages
    printf("The error generated was %d\n", errvalue ); // as an integer
    printf("That means: %s\n", strerror( errvalue ) ); // translated to an error msg str
    printf("Main terminated\n");
}
```

Console Output:



```
<terminated> Sample01-Beagle06 [C/C++ QNX QConn (IP)] /t
The error generated was 9
That means: Bad file descriptor
Main terminated
```

**This line will
cause an error**

errno List

- The **errno** variable can be translated in to an error message string by using **strerror()** function, which supported functions can use.
- There are over 130 error strings. **Note: NOT** all **errno** values are POSIX compliant.

Name	Value	Meaning	POSIX?
E2BIG	7	Argument list is too long	Yes
EACCES	13	Permission denied	Yes
EADDRINUSE	248	Address is already in use	Yes
EADDRNOTAVAIL	249	Can't assign requested address	Yes
EADV	68	Advertise error	No
EAFNOSUPPORT	247	Address family isn't supported by protocol family	Yes
EBADE	50	Invalid exchange	No
EBADF	9	Bad file descriptor	Yes
EBADFD	81	FD is invalid for this operation	No
EBADFSYS	302	Corrupted filesystem detected	No
EBADMSG	77	Bad message	Yes
EBADR	51	Invalid request descriptor	No
EBADRQC	54	Invalid request code	No
EBFONT	57	Bad font-file format	No
EBUSY	16	Device or resource is busy	Yes
ECANCELED	47	Operation canceled	Yes
ECHILD	10	No child processes	Yes
ECHRNG	37	Channel number is out of range	No

The full list can be found:

http://www.qnx.com/developers/docs/7.0.0/index.html#com.qnx.doc.neutrino.lib_ref/topic/e/errno.html

NOTE: These lecture notes will not be checking the return value of function calls or the **errno** variable to keep the code in the notes as readable as possible.

Design Question: Joinable vs. Detached Threads

- By default all threads in QNX are joinable.
 - You can create detached threads using the thread attributes before spawning it.
 - You can also use `pthread_detach()` to detach a thread from a joined process.
- The detached thread will still have the same process ID as the parent thread
- Detached threads should be used when you know you won't want to wait for it with `pthread_join()`.
 - Performance benefit of a detached threads are:
 - The allocated system resources are immediately released to the OS when they terminate.
 - Joined threads only release their resources after they have been joined – They sit there in a **blocked** state.
- It is **'legal'** not to join a joinable thread; but it is not usually advisable because the system resources won't be released until the thread is joined, so they'll remain tied up indefinitely (until the program exits) if you don't use `pthread_join()` or equivalent.

```
#include <pthread.h>

int pthread_detach( pthread_t thread_ID );
```



The ID of the thread that you want to detach, which you can get when you call `pthread_create()` or `pthread_self()`

Detached Thread Example

- Example of joined thread and detached threads.
 - By changing the sleep time in `main()` to `<5` and `>5` it is possible see how the threads behave differently in each example.

Create a Joined Thread

```
void *thread_ex(void *not_used)
{
    printf("Thread Started\n");
    sleep(5); // do something
    printf("Thread Finished\n");
}

void main()
{
    pthread_t  th1;

    pthread_create(&th1, NULL, thread_ex, NULL);
    sleep(10); // wait here longer than needed

    pthread_join(th1, NULL); // block here
    printf("Main terminated\n");
}
```



Create a Detached Thread

```
void *thread_ex(void *not_used)
{
    printf("Thread Started\n");
    sleep(5); // do something
    printf("Thread Finished\n");
}

void main()
{
    pthread_t  th1; // Not required for this Ex
    pthread_attr_t att;

    pthread_attr_init( &att );
    pthread_attr_setdetachstate(&att,
                                PTHREAD_CREATE_DETACHED );
    pthread_create(NULL, &att, thread_ex, NULL);
    sleep(6); // let thread run for 6 sec

    printf("Main terminated\n");
}
```


Detaching a Joined Thread at Runtime

- Even though a thread is created as a Joined thread, it is possible to detach it at runtime. There are 2 options:

```
void *thread_ex(void *not_used)
{
    printf("Thread Started\n");
    sleep(5); // do something
    pthread_detach( pthread_self() );
    printf("Thread Finished\n"); // detach thread here
}

void main()
{
    pthread_t  th1;
    pthread_create(&th1, NULL, thread_ex, NULL);

    //pthread_detach(th1); // Can be detached here too

    sleep(10); // wait here longer than needed

    int retcode = pthread_join(th1, NULL); // block here
    if(retcode == EOK)
        printf("Thread Successfully Joined\n");
    else
        printf("Error %i: %s\n", retcode, strerror(retcode));
    printf("Main terminated\n");
}
```

```
#include <pthread.h>
pthread_t pthread_self( void );
```

Returns the thread ID of the calling thread as type `pthread_t`
Just a special type of `int`

Thread Attributes

- Thread attributes are defined by 2 structures
 - `sched_param`
 - `pthread_attr_t`
- Used to control the "Scheduling policies"

```
void *thread_ex(void *not_used) {
    printf("Thread Started\n");
    sleep(4); // do something
    printf("Thread Finished\n"); }
```

```
void main() {
    pthread_t  th1;
    pthread_attr_t th1_attr;      // Thread attributes
    struct sched_param th1_param; // Scheduling parameters

    // Init thread attribute to default values (required)
    pthread_attr_init(&th1_attr);
    // set scheduling to round robin
    pthread_attr_setschedpolicy(&th1_attr, SCHED_RR);
    //Set thread priority (can be from 1..63, default=10)
    th1_param.sched_priority = 1;
    pthread_attr_setschedparam(&th1_attr, &th1_param);

    //Now set attribute to use the explicit scheduling settings (must also do this)
    pthread_attr_setinheritsched(&th1_attr, PTHREAD_EXPLICIT_SCHED);
    pthread_create(&th1, &th1_attr, thread_ex, NULL); //Create and start the thread

    pthread_join(th1, NULL); //Block her until thread finishes
    printf("Main terminated\n"); }
```

```
#include <sched.h>
struct sched_param {
    int32_t  sched_priority;
    int32_t  sched_curpriority;
    union {
        int32_t  reserved[8];
        struct {
            int32_t  __ss_low_priority;
            int32_t  __ss_max_repl;
            struct timespec __ss_repl_period;
            struct timespec __ss_init_budget;
        } __ss;
    } __ss_un;
}
```

for SCHED_SPORADIC

```
typedef struct {
    int      __flags;
    size_t   __stacksize;
    void     *__stackaddr;
    void (*__exitfunc)(void *status);
    int      __policy;
    struct sched_param __param;
    unsigned __guardsize;
} pthread_attr_t;
```

Getting and Setting Thread Priority and Schedule

Include `#include <sched.h>` or `#include <pthread.h>` for the following functions:

- To get the current priority level of a process:

```
int sched_getparam( pid_t pid, struct sched_param *param );
```

- Get the current scheduling policy for a process:

```
int sched_getscheduler( pid_t pid );
```

- Set `pid` to `0` if you want to the policy for the current process
- `1 = SCHED_FIFO`, `2 = SCHED_RR`, `3 = SCHED_OTHER`

- Find out the MAX or Min priority for a scheduling policy

```
int sched_get_priority_max( int policy );
```

```
int sched_get_priority_min( int policy );
```

`policy` be one of: `SCHED_FIFO`, `SCHED_RR`, `SCHED_OTHER` or `SCHED_SPORADIC`

- Change the priority and scheduling policy of a process:

```
int sched_setscheduler( pid_t pid, int policy, const struct sched_param *param );
```

- Change the priority of a process:

```
int sched_setparam( pid_t pid, const struct sched_param *param );
```

- The structure `sched_param` describes scheduling parameters



```
struct sched_param {
    int32_t sched_priority;
    int32_t sched_curpriority;
    union {
        // ... See slide 22
        // for full content
        // these must be used
        // for SCHED_SPORADIC
    } __ss;
} __ss_un;
```

Getting and Setting Thread Priority and Schedule Example

Example shows how to read scheduling policies and change them at runtime:

```
void Get_thread_params(struct sched_param *p, int size)
{
    memset(p, '\0', size);           // clear param structure
    sched_getparam(0, p);           // Find out what the current priority is
    // Find out what the scheduling policy for the current process is
    int sched_policy = sched_getscheduler(0);
    printf("The assigned scheduling policy for this process is %d.\n", sched_policy);
    printf("The assigned priority is %d.\n", p->sched_priority);
    printf("The current priority is %d.\n", p->sched_curpriority);
}

void main (void)
{
    struct sched_param param;        // declare a param structure
    Get_thread_params(&param, sizeof(param));
    // Change the scheduling policy FIFO with a priority of 15
    param.sched_priority = 15;
    sched_setscheduler(0, SCHED_FIFO, &param);
    Get_thread_params(&param, sizeof(param));
    printf("Main terminated\n");
}
```

More complete version with error checking:

sched_policy.c

Passing Data Structures between Threads

- So far we have shown that it is possible to pass data (an `int`) to a thread using the last argument of `pthread_create(NULL, NULL, &thread_routine, &arg)`
- By passing the address of a `struct` it is possible pass more than 1 variable to a thread.

```
typedef struct {
    int var1;
    int var2;
    int var3;
} data_bundle;

void main( void )
{
    data_bundle dat = {5, 6, 210};

    pthread_t th1;
    pthread_create(&th1, NULL, thread_ex, &dat);
    pthread_join (th1, NULL);

    printf(" var1=%d\n var2=%d\n var3=%d\n",
           dat.var1, dat.var2, dat.var3);
    printf("Main Terminated...\n");
}
```

```
void *thread_ex(void *data)
{
    data_bundle *a = (data_bundle*)data;

    a->var1 = 12; // modify the
    a->var2 = 22; // data values
    a->var3 = 32;

    printf("Thread Finished\n");
}
```



Type casting is used here to cast the `void *` pointer to a `struct` of type `data_bundle`.

There is no fixed requirements for the data held in the `struct`.

The `struct` can also contain nested data structures or unions.

Interrupt

Interrupt Handling

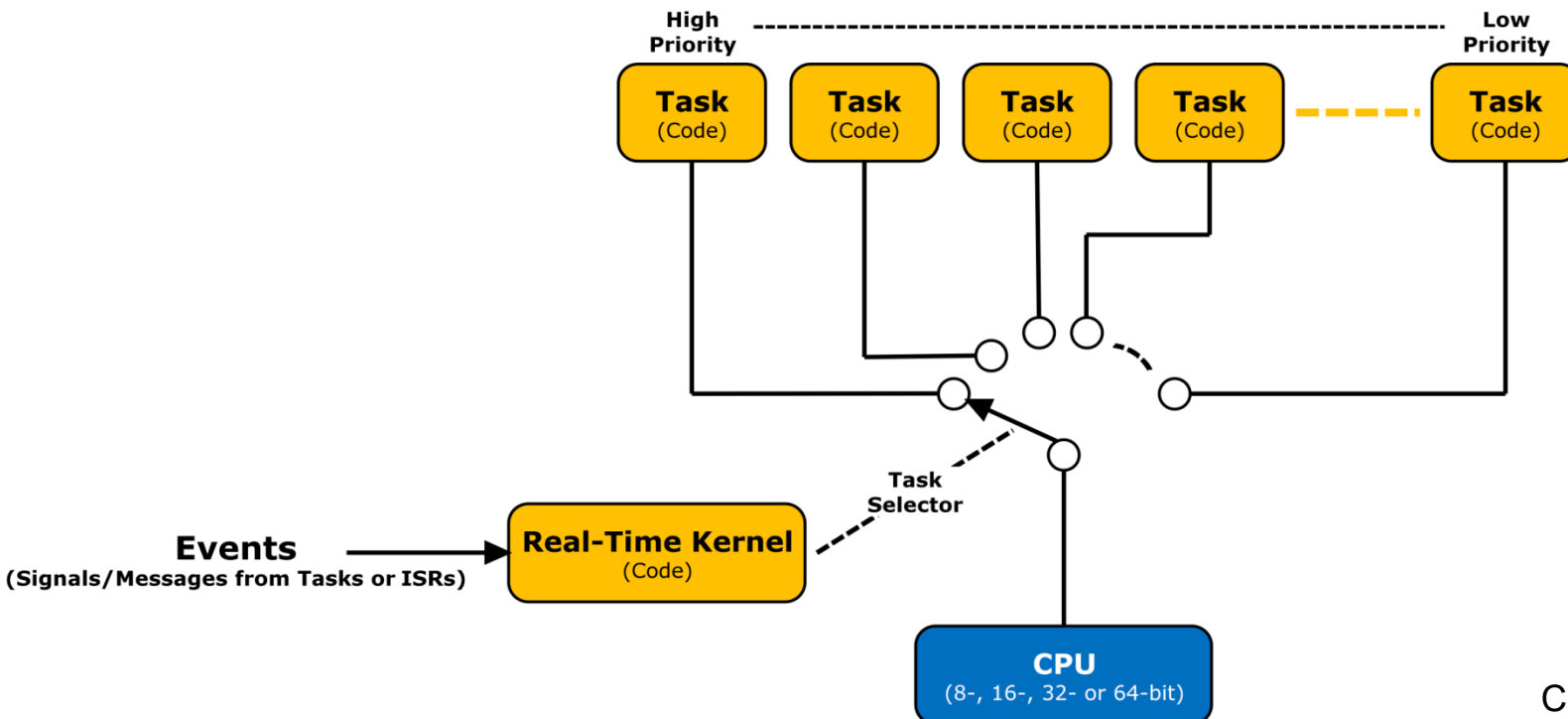
- A **Context Switch** on interrupt
 - Save values of **CPU registers** in the process control block of the running process, and switch to the context of the interrupt service routine
 - e.g. May involve changing memory management mapping and changing privilege level
 - When interrupt service is completes the system may resume the original process (context switch) or switch to another process (process switch)
- An interrupt service routine is similar to a lightweight process even if there is no operating system involved. e.g. For an embedded controller
- **NOTE:** ISR's can only execute a limited number of instructions and has a significantly reduced stack size.



[3]

Process Switch

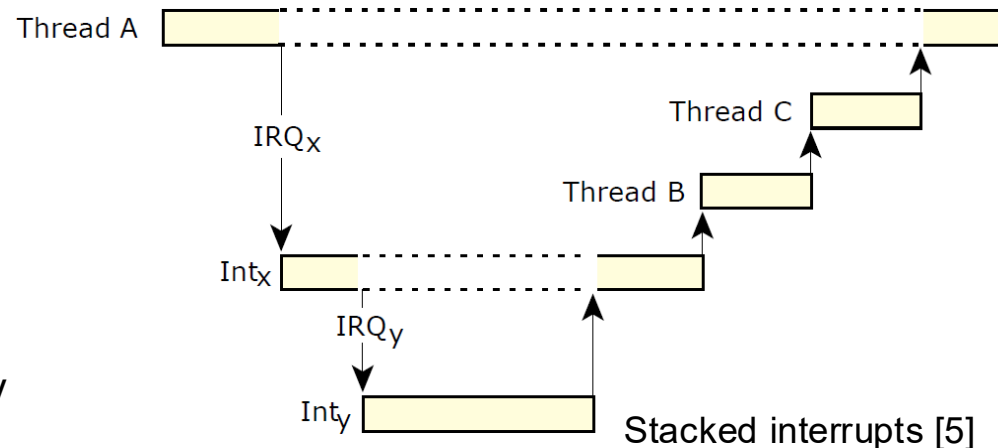
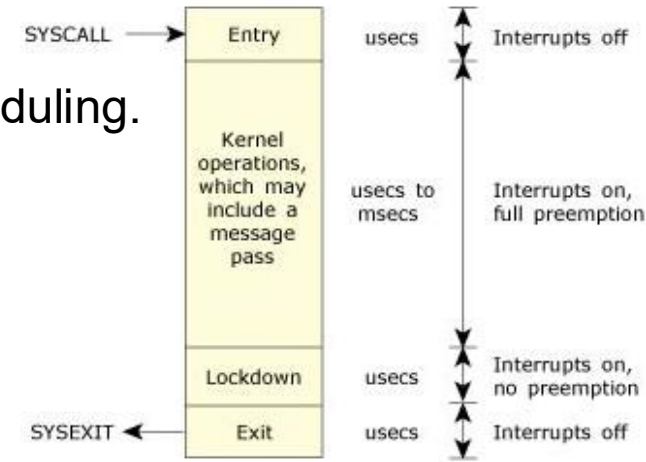
- A **Process Switch** is more complex than a **context switch**.
 - It involves two context switches.
 1. Move old running process to the tail of the ready queue (or to a blocked queue)
 2. Select new running process and dispatch it (Usually the highest priority ready thread)
- RT systems try very hard to minimize the time that interrupts are disabled
 - An Interrupt service routine should only take action to remove the cause of the interrupt and then schedule a high priority operating system task to handle the interrupt.
 - Hence an operating system will usually have multiple concurrent processes (tasks)



Context switching [4]

Preemption

- Most RTOS perform priority-based preemptive task scheduling.
 - Every task in the system is assigned a priority
 - Higher priority = Higher need for Quick response
 - Overarching RULE:**
 - The highest priority task that is ready to run will be the task that **MUST** be running!
 - QNX manual state: *“Interrupts are disabled, or preemption is held off, for only very brief intervals (typically in the order of hundreds of nanoseconds).”*
- Priority based preemptive scheduler goes through the following process every time an **external world trigger (IRQ)** or **software trigger occurs**:
 - Determine whether the current task running should continue to run
 - Determine which task should run next
 - Save the environment of the task that was stopped (so it can continue later).
 - Set up the running environment of the task that will run next.
 - Allow the selected task to run.
- A thread is preempted when a higher-priority thread is placed on the ready queue.



Multiple CPU

Thread affinity

- Using thread affinity it is possible for different threads to be assigned to a specific CPU core within a multicore node.

```
#include <sys/neutrino.h>

int ThreadCtl (int masks_cmd, void * struct _thread_runmask);
```

- QNX provides the ability to associate certain programs (or even threads within programs) with a particular core or processor.
- When a thread starts up, its affinity mask (or runmask) is set to allow it to run on all available processors.
- Threads can be made to inherit their properties from the parent thread/process.
- You can use **_NTO_TCTL_RUNMASK_GET_AND_SET_INHERIT** to optimize the runtime performance of your system by, for example, relegating nonrealtime threads to a specific processor and make all child threads inherit this setting.
 - In general, this shouldn't be necessary, since the QNX realtime scheduler always preempts a lower-priority thread immediately when a higher priority thread becomes ready.
- The main effect of processor locking is the effectiveness of the CPU cache, since threads can be prevented from migrating.

StressTestCPU.c**CPU_Usage.c**

Speed Test – Does multithreading make a difference??

- 4 ARM targets each with different configuration
 - BeagleBone, 1 Core, 720 MHz, A8
 - DE10-nano, 2 Core, 800 MHz, A9
 - Beagle X15, 2 Core, 1000 MHz, A15
 - iMX.6, 4 Core, 792 MHz, A9
- Targets have to calculate π to 50,000,000 decimal place 5 times (separate threads)
- Speed test results:

Target	Single Core (seconds)	All Cores (seconds)	MHz
BeagleBone	112.32	112.31	720
DE10-nano	22.93	11.48	800
iMX.6	23.13	5.80	792
Beagle X15	9.04	4.52	1000

- The DE10 and the iMX.6 have very comparable core speed and are both cortex-A9, however the iMX.6 has 4 cores, rather than the 2 core DE10
 - iMX.6 is ~2 times faster.
- A15 architecture is much faster than the A8/A9 variants

Multi Threaded Factorial Example

- Threads can be used to parallelize code to run on multiple core-CPU.
 - The below program uses a threads to calculate the Factorials
 - Creating threads does have an overhead cost (in this example about **double** to **triple** the time it would take if it was written to execute sequentially).

```
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#include <sys/syspage.h>
#include <inttypes.h>      // for clock cycles calculation
#include <sys/neutrino.h>  // for clock cycles calculation

#define dsize 21          // data source size

void *factorial_ex(void *data)
{
    // Input number we are using for the calculation
    unsigned long long num = *((unsigned long long*)data);

    // variable we are storing the answer in
    unsigned long long factorial = 1;

    // Do the calculation
    int i = 0;
    for(i = 1; i <= num; i++)
        factorial = factorial * i;

    printf("ThreadID:%d\t %llu! = %llu\n", getpid(), num, factorial); // Print answer
}
```


Multi Threaded Factorial Example

- The `main()` function that creates all the threads:

```
void main( void )
{
    pthread_t *thread_ids;           // Create space to thread ID pointers
    thread_ids = malloc (sizeof (pthread_t) * dsize); // clear the space
    uint64_t cps, cycle1, cycle2, ncycles; // Define variables used for timing
    double sec;

    int i =0, th_launched =0;
    unsigned long long data[dsize] = {}; // Create space to hold data
    for(i=0;i < dsize; i++) // Setup data to preform operation on
        data[i] = i;

    cycle1=ClockCycles(); // Get Start TIME
    for(i=0;i < dsize; i++, th_launched++) // Do the actual calculations
        pthread_create(&thread_ids[th_launched], NULL, factorial_ex, &data[i]);
    for(i=0; i<th_launched; i++) // Wait here to join all threads
        pthread_join(thread_ids[i], NULL);

    cycle2=ClockCycles(); // Get End TIME and preform time calc
    ncycles=cycle2-cycle1;
    cps = SYSPAGE_ENTRY(qtime)->cycles_per_sec;
    sec=(double)ncycles/cps * 1000;
    printf("This system has %lld cycles/sec.\n",cps );
    printf("Computation time = %f milliseconds.\n",sec);
}
```


Speed Test – Does multithreading make a difference??

- 4 ARM targets each with different configuration
 - BeagleBone, 1 Core, 720 MHz, A8
 - DE10-nano, 2 Core, 800 MHz, A9
 - Beagle X15, 2 Core, 1000 MHz, A15
 - iMX.6, 4 Core, 792 MHz, A9
- Targets have to calculate factorial of each number up until 20 (with/without threads)
- Will the multithreaded code be faster than single core execution?
- Speed test results:

Target	Threaded (milliseconds)	Sequential (milliseconds)	MHz
BeagleBone	13.89	6.18	720
DE10-nano	3.93	2.63	800
iMX.6	4.37	2.67	792
Beagle X15	3.01	2.30	1000

- Why is the Threaded version of the code slower in all cases?

IPC

Interprocess Communication (IPC)

RMIT Classification: Trusted

- RT systems generally require concurrently running threads to have a **safe way to communicate between each other**.
- In the case of QNX, message passing lies at the heart of the operating system's microkernel architecture, giving the OS its modularity.
 - Most OS provide mechanisms to allow threads to communicate
 - You may have used a file to store useful information between 2 or more processes
 - QNX uses Interprocess Communication (IPC)
 - IPC plays a fundamental role in the transformation of the QNX microkernel from an embedded real time kernel into a **full-scale POSIX operating system**.
- Simple Message passing as fundamental means of IPC, which allows:
 - Process and threads to pass data to each other
 - Messages (parcel of bytes, i.e. a string)
 - Information exchange (i.e. numbers, etc.)
 - IPC Provide a way to synchronize the execution of several processes
 - Over distributed networks or within the same node
- Mission-critical applications often require dependable forms of IPC as their processes are generally strongly interrelated.

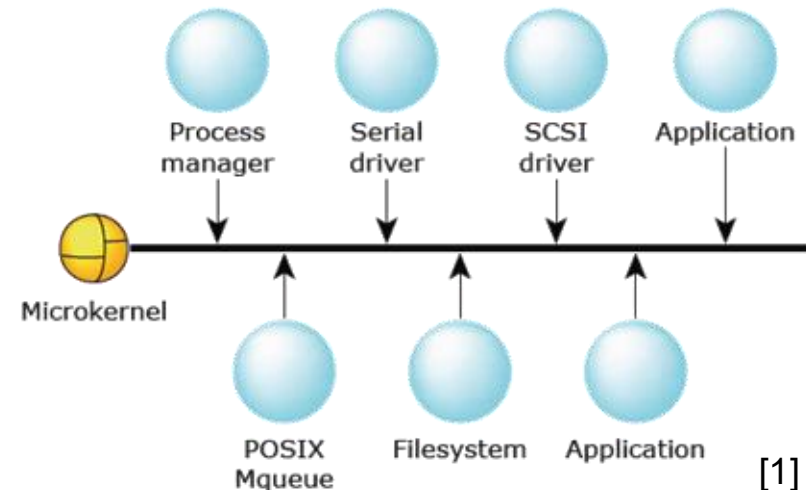


Interprocess Communication (IPC)

- QNX offers the following forms of IPC:

Service:	Implemented in:	
⇒ POSIX Message queues	External process	Lab 2
⇒ Signals	Kernel	Lab 4
⇒ Native Message-passing	Kernel	Lab 5
Shared memory	Process manager	
Pipes	External process	
FIFOs	External process	

- IPC is often more efficient than network-distributed shared memory
 - Shared memory generally requires a minimum copy size (4K Bytes)
 - Messaging can be quiet tiny (eg. Error status, write request, couple bytes of data)
- The designer can select these services on the basis of bandwidth requirements, the need for queuing, network transparency, etc.
 - Setup can be complex, but is highly flexible



[1]

POSIX Message Queues (in QNX)

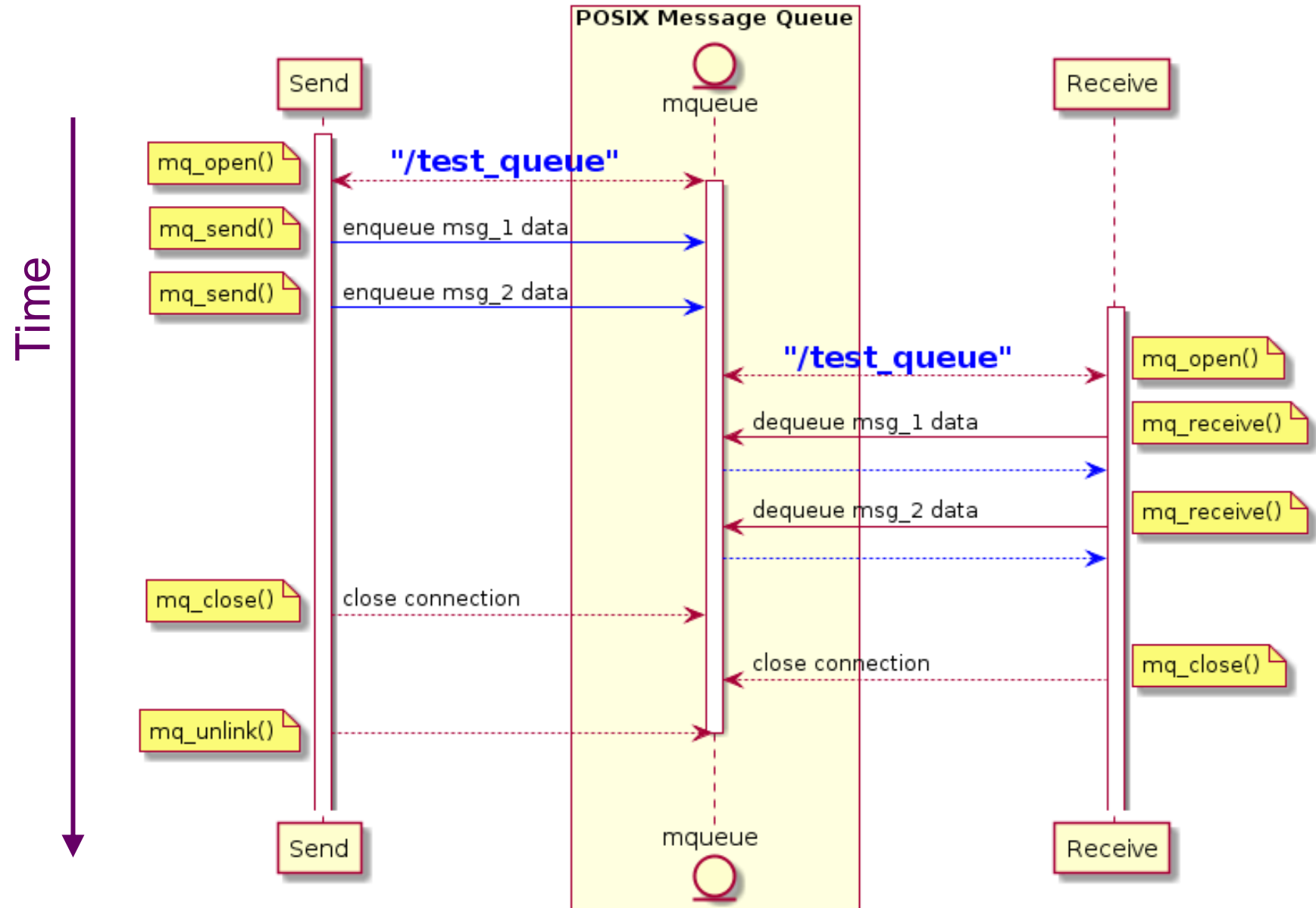
- POSIX message queues provide a standard interface between compliant RTOS.
 - They employ a **Mailbox** approach found in many real-time executives.
 - store-and-forward design in which **the sender does not block** (unless the queue is **full**).
 - Data is directly copied between the address spaces of the processes sending the messages to the POSIX message queues, which runs independently to send or receive process.
- To establish a message queue, first the server side code must create a mqqueue within A QNX nodes filesystem – does not need to be the same node

```
mqd_t fd1, fd2;  
  
fd1 = mq_open( const char * name, int oflag, int mode, mq_attr &attr); // new queue  
  
fd2 = mq_open( const char * name, int oflag );           // if the queue already exists
```

Arguments:

- | | |
|----------------|--|
| name | The name of the message queue resource created in the filesystem |
| oflag | O_RDONLY (<i>receive-only</i>), O_WRONLY (<i>send-only</i>) or O_RDWR (<i>send-receive</i>).
can be OR with O_CREAT, O_EXCL (fail if created) and O_NONBLOCK (no block if full) |
| mode | file permissions for the new queue (owner and group permissions) |
| mq_attr | pointer to a structure that contains the attributes of the queue (size, bytes, etc.) |

POSIX Message Queues (in QNX)



Sequence Diagram generated with plantuml.com

POSIX mqueues – Send Process

```

void main( void ) {
    mqd_t qd;                // Declare mqueue object
    char buf[MESSAGESIZE] = {}; // buffer
    int i = 0;

    struct mq_attr attr;      // struct containing message queue's attributes
    attr.mq_maxmsg = 100;     // Maximum number of messages
    attr.mq_msgsize = MESSAGESIZE; // Maximum size of a message
    attr.mq_flags = 0;
    attr.mq_curmsgs = 0;      // If you are changing one member of this struct you need
    attr.mq_sendwait = 0;     // make sure all other struct members are defined !!!!
    attr.mq_recvwait = 0;     // The other option is to NULL for the attr arg in mq_open()

    qd = mq_open("/test_queue", Q_FLAGS, Q_Mode, &attr); // Open a connection at /test_queue
    if(qd != -1)
    {
        for(i=1; i <= 5; ++i)
        {
            sprintf(buf, "message %d", i); // Create the message packet
            printf ("Send to mqueue: '%s'\n", buf); // print message being sent to console
            mq_send(qd, buf, MESSAGESIZE, 0); // send message packet
            sleep(5);
        }
        mq_send (qd, "done", 5, 0); // send msg to tell client we are all done
        mq_close (qd); // Close the connection
        mq_unlink("/test_queue"); // Remove a message queue
    }
    printf("Send Process Terminated...\n");
}

```

```

#include <stdlib.h>
#include <stdio.h>
#include <mqueue.h>
#include <sys/stat.h>
#define MESSAGESIZE 1000
#define Q_FLAGS O_RDWR | O_CREAT
#define Q_Mode S_IRUSR | S_IWUSR

```


POSIX mqueues – Receive Process

- Send process should be start prior to evoking the Receive process, otherwise the mqueue will not exist.

```
#include <stdio.h>
#include <mqueue.h>

#define MESSAGE_SIZE 1000
```

```
void main( void )
{
    mqd_t qd;                // Declare mqueue object
    char buf[MESSAGE_SIZE] = {}; // buffer

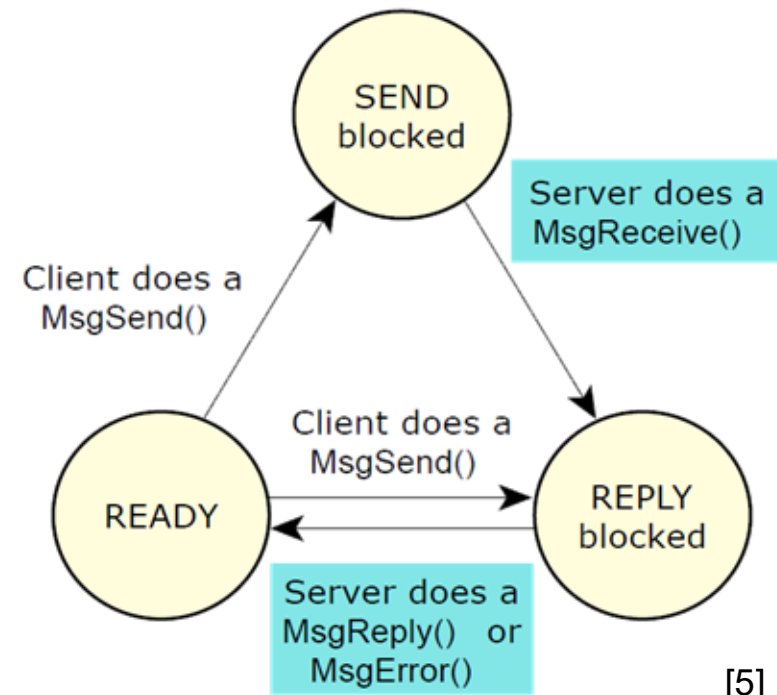
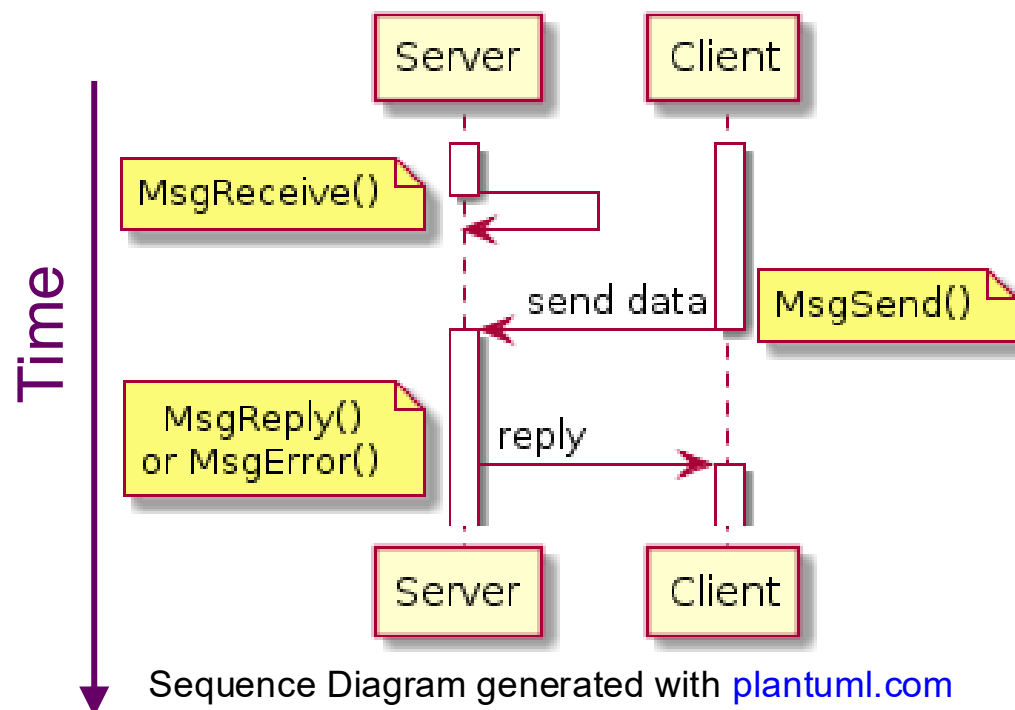
    struct mq_attr attr;     // struct containing message queue's attributes

    qd = mq_open("/test_queue", O_RDONLY); // Open a connection to a known location
    if(qd != -1)
    {
        mq_getattr(qd, &attr); // Get the attributes of the mqueue (from the sender)
        printf ("max. %u msgs, %u bytes; waiting: %u\n",
                attr.mq_maxmsg, attr.mq_msgsize, attr.mq_curmsgs);

        while (mq_receive(qd, buf, MESSAGE_SIZE, NULL) > 0)
        {
            printf("Received from mqueue: '%s'\n", buf);
            if (!strcmp(buf, "done")) // Check if the message indicates we got the last msg
                break; // "done" is the expected value for the last msg
        }
        mq_close(qd); // Close message queue connection
    }
    printf("Receive Process Terminated...\n");
}
```

Native QNX Message Passing and the Client/Server Model

- QNX Message passing enables scalability and network-distributed processing with far less effort than other OS's. It also provides the following capabilities:
 - Information sharing - Resource sharing - Computation speedup
 - Synchronization - Modularity - **More efficient than POSIX implementation**
- Native QNX messages are *always* delivered in priority order
 - Can lead to priority inversions due to their blocking nature.
 - Sequence & State Diagram of a sender-driven messaging Client/Server



[5]

Native QNX Message Passing

- The process of sending a message is very similar to opening a file and writing a block of data to it.
- 3 distinct messages are being sent:
 - `open()` sent an “open” message
 - `write()` sent a “write” message
 - `close()` sent a “close” message
- These messages are being sent from the application to the installed filesystem component on the OS – remember everything in QNX is a service.
 - message passing is done *very* quickly, with little overhead.
 - On a traditional OS, the `open()` function call would invoke a kernel function, which would then call directly into the filesystem, which would execute some code, and return a file descriptor. Similar process for `write()` and `close()` calls.
 - If the file was networked, then remote (networked) filesystem calls need to be made.
- As QNX is a network-distributed OS based fundamentally on message passing principles, all we need to do to write a file on another node’s filesystem is change the `open()` line to:


```
fd = open("/net/NodeName/home/rk/filename.txt", O_WRONLY);
```

 - The file descriptor *is just* a connection ID! – minimized overhead compared to other OS

```
#include <fcntl.h>
#include <unistd.h>

void main( void )
{
    int fd;
    fd = open("filename", O_WRONLY);
    write(fd, "This is message passing\n", 24);
    close(fd);
    printf("Main Terminated...\n");
}
```

Using Native QNX Message Passing Between Threads

- QNX Native Message Passing is based on a client-server model where the server must run and open a channel on which to accept messages.
- Basic pathname-to-server-connection mapping, without having to become a full resource manager following this process:
 - The server thread invoke `name_attach()`
 - registers a name under `"/dev/name"`
 - The client then uses `name_open()` to connect
 - The server then calls `MsgReceive()` (determine if receiving a pulse or a message)
 - The server stays blocked until a client invokes `MsgSend()`
 - The client stays blocked until the server invokes `MsgReply()`
 - `name_close()` and `name_detach()` are used by the client and server, respectively to close down the connection.

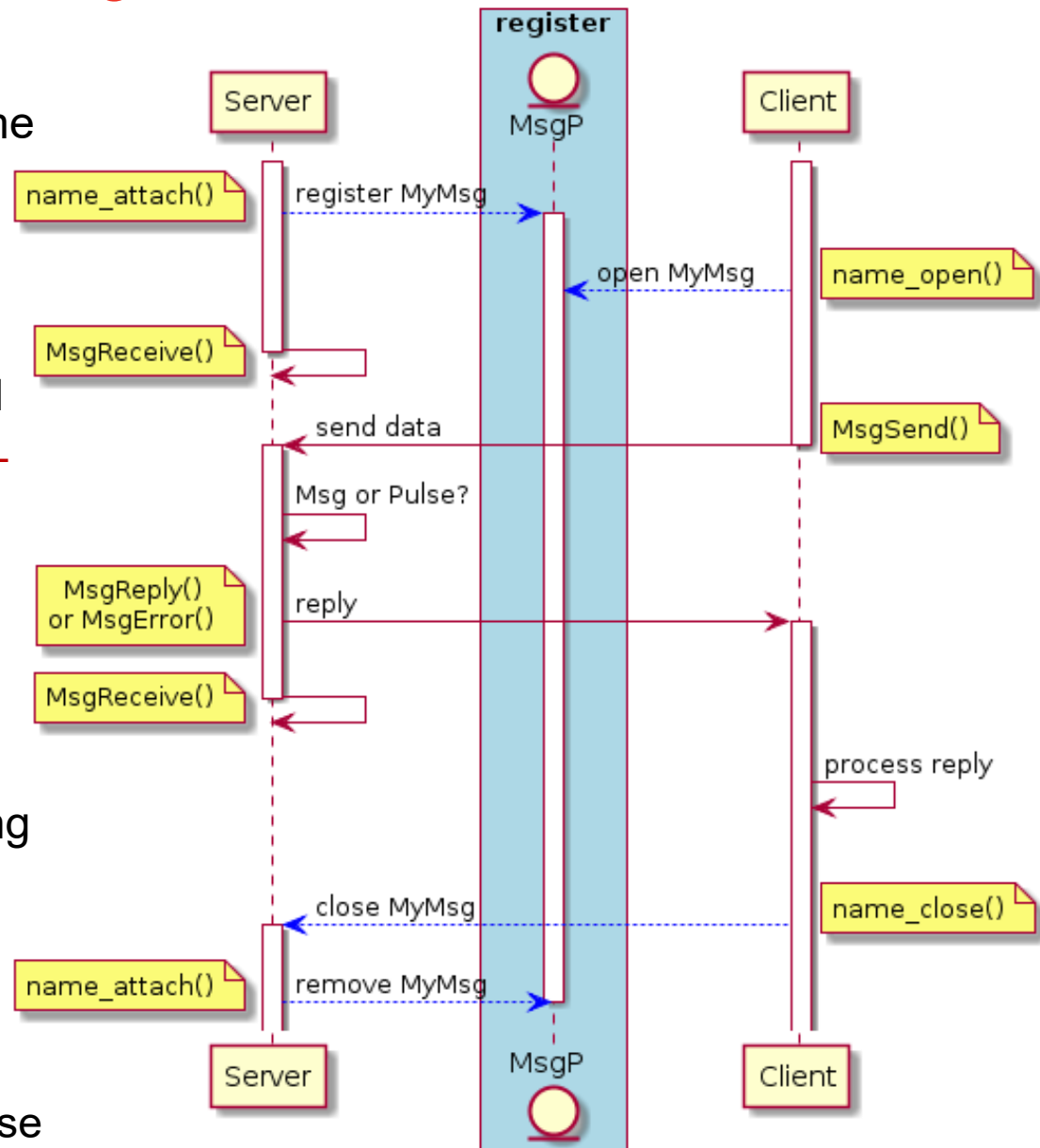
```
name_attach_t *attach;  
attach = name_attach( dispatch_t *dpp, const char *path, unsigned int flags );
```

Arguments:

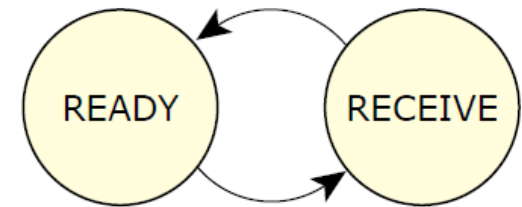
- | | |
|--------------|--|
| dpp | NULL, or a dispatch handle returned by a successful call to <code>dispatch_create()</code> or <code>dispatch_create_channel()</code> . |
| path | The name that will be registered under the path <code>/dev/name/[local global]/</code> |
| flags | 0, or flags that affect the function's behavior: ie <code>NAME_FLAG_ATTACH_GLOBAL</code> |

Native QNX Message Passing and Pulses

- A **server process** can use `name_attach()` to register a name (and create a channel)
 - The name is put into the namespace under `/dev/name/local/` or `/dev/name/global/` if attached with `NAME_FLAG_ATTACH_GLOBAL` flag set
- A **client process** can use `name_open()` to obtain a connection ID for sending to the server
- **Pulses** are fixed-size, non-blocking message (shown in blue)
 - Four bytes of data plus a single byte code
 - Non-blocking for sender
 - Receive but do not reply to a pulse

generated with plantuml.com

QNX Msg Passing – Server Thread



State transitions of server [1]

```

void main( void )
{
    name_attach_t *attach;
    my_data msg;                // message structure
    my_reply replymsg;          // replymsg structure for sending back to client
    replymsg.hdr.type = 0x01;    // some number to help client process reply msg
    replymsg.hdr.subtype = 0x00; // some number to help client process reply msg
    int rcvid=0, msgnum=0;      // no message received yet

    attach = name_attach(NULL, ATTACH_POINT, 0); // create the connection
    if(attach != NULL)
    {
        printf("Server Listening for Clients on ATTACH_POINT: %s \n", ATTACH_POINT);
        while(1)
        {
            rcvid = MsgReceive(attach->chid, &msg, sizeof(msg), NULL);
            if(rcvid == 0)                // A Pulse was received, workout what type
            {
                printf("Received a PULSE of type: %d\n", msg.hdr.code);
                if(msg.hdr.code == _PULSE_CODE_DISCONNECT)
                {
                    printf("Received detach PULSE - Bye!\n");
                    ConnectDetach(msg.hdr.scoid);
                    break;
                }
            }
            else if(rcvid > 0)            // A message was received, workout what type
            {
                msgnum++;                // keep track of how many message we receive
                printf("Received a MESSAGE of type: %d, ", msg.hdr.type);
                printf("with value of '%d' from client (ID:%d)", msg.data, msg.ClientID);
                sprintf(replymsg.buf, "Message %d received", msgnum);
                sleep(1); // Delay the reply by a second (just for demonstration purposes)
                printf("-> replying with: '%s'\n", replymsg.buf);
                MsgReply(rcvid, EOK, &replymsg, sizeof(replymsg));
            }
            else if(rcvid == -1)          // Error condition, exit
            {
                printf("\nFailed to MsgReceive\n");
                break;
            }
        }
        name_detach(attach, 0);
    }
    printf("Main (Server) Terminated....\n");
}
  
```

MsgPass-Server.c

Register name– `name_attach()`

Receive message – `MsgReceive()`

`if(rcvid == 0)`
 Bit that handles any **Pulses** sent by a client or the kernel. These should be decoded and handled accordingly:

- `_PULSE_CODE_DISCONNECT`
- `_PULSE_CODE_UNBLOCK`
- `_PULSE_CODE_THREADDEATH`

`if(rcvid > 0)`
 Bit that handles any **Messages** sent by a client or the kernel. These should be decoded and handled accordingly:

- `_IO_CONNECT` (GNS server)
- `_IO_BASE` (IO message)

Receive message – `MsgReply()`

`if(rcvid == -1)`
 An **error** occurred

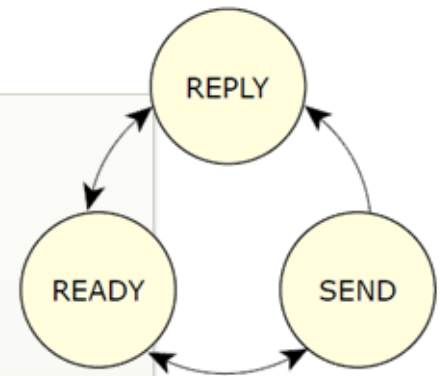
QNX Msg Passing – Client Thread

RMIT Classification: Trusted

```
void main( void )
{
    my_data msg;
    my_reply reply;
    msg.ClientID = 600;        // unique number for this client
    msg.hdr.type = 0x22;       // We would have pre-defined data to stuff here
    msg.hdr.subtype = 0x00;    // more stuff we can define
    int server_coid=0, i=0;

    printf(" ---> Trying to connect to server named: %s\n", ATTACH_POINT);
    server_coid = name_open(ATTACH_POINT, 0); // Open connection
    if(server_coid != -1)
    {
        printf("Connection established to: %s\n", ATTACH_POINT);
        for(i=0; i < 5; i++) // send data packets
        {
            msg.data = 10+i;    // Make some data to send
            // the data we are sending is in msg.data
            printf("Client (ID:%d), sending data: %d ", msg.ClientID, msg.data);

            if(MsgSend(server_coid, &msg, sizeof(msg), &reply, sizeof(reply)) == -1)
            {
                printf(" Error sending data '%d' to server\n", msg.data);
                // maybe we did not get a reply from the server
                break;
            }
            else
            {
                // now process the reply
                printf(" --> Reply from server is: '%s'\n", reply.buf);
            }
            sleep(1);           // wait a second before sending the next data packet
        }
        printf("Client (ID:%d), sending PULSE requesting server to close connection\n", msg.ClientID);
        name_close(server_coid);
    }
    printf("Main (Client) Terminated....\n");
}
```



State transitions of client [1]

Open connection –
`name_open()`

Loop that sends 5 messages

`MsgSend()` - Send the
message and **Block** until
server sends a reply msg

Do something with the reply

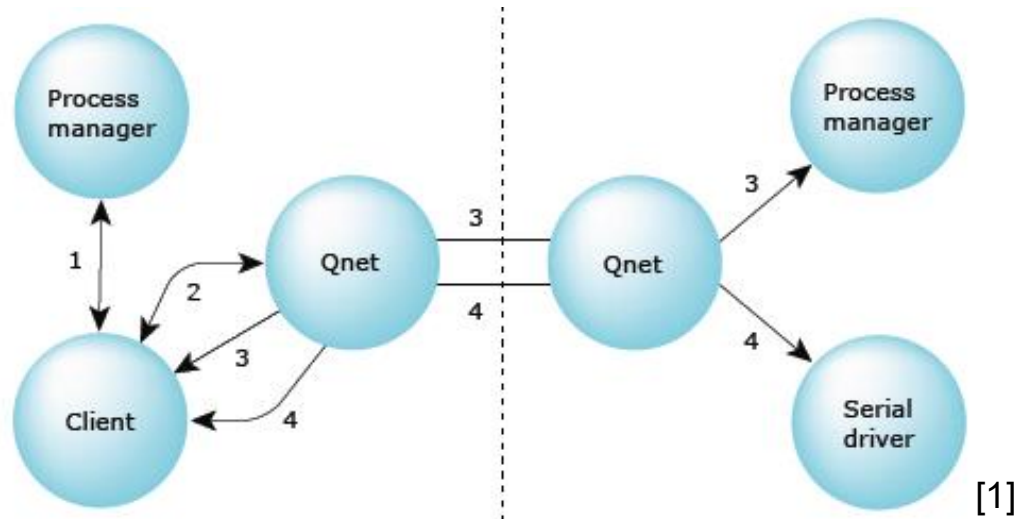
Send a pulse to close server -
`name_close()`

MsgPass-Client.c

Message passing over a network

- QNX is designed from the ground up as a network-wide OS
 - The aim is to provide an environment where there is little to no difference between a local or remote resources
 - Access files or use peripheral devices anywhere on a network
 - Run applications on other nodes
 - Process communicated in the same manner anywhere throughout the network
 - The IPC services provide transparent networking
- QNX nodes can see each other if the **qnet** service is loaded into memory. If you don't have a **/net** directory in your target's file system then you can load **qnet** by typing the following command at the QNX terminal prompt:

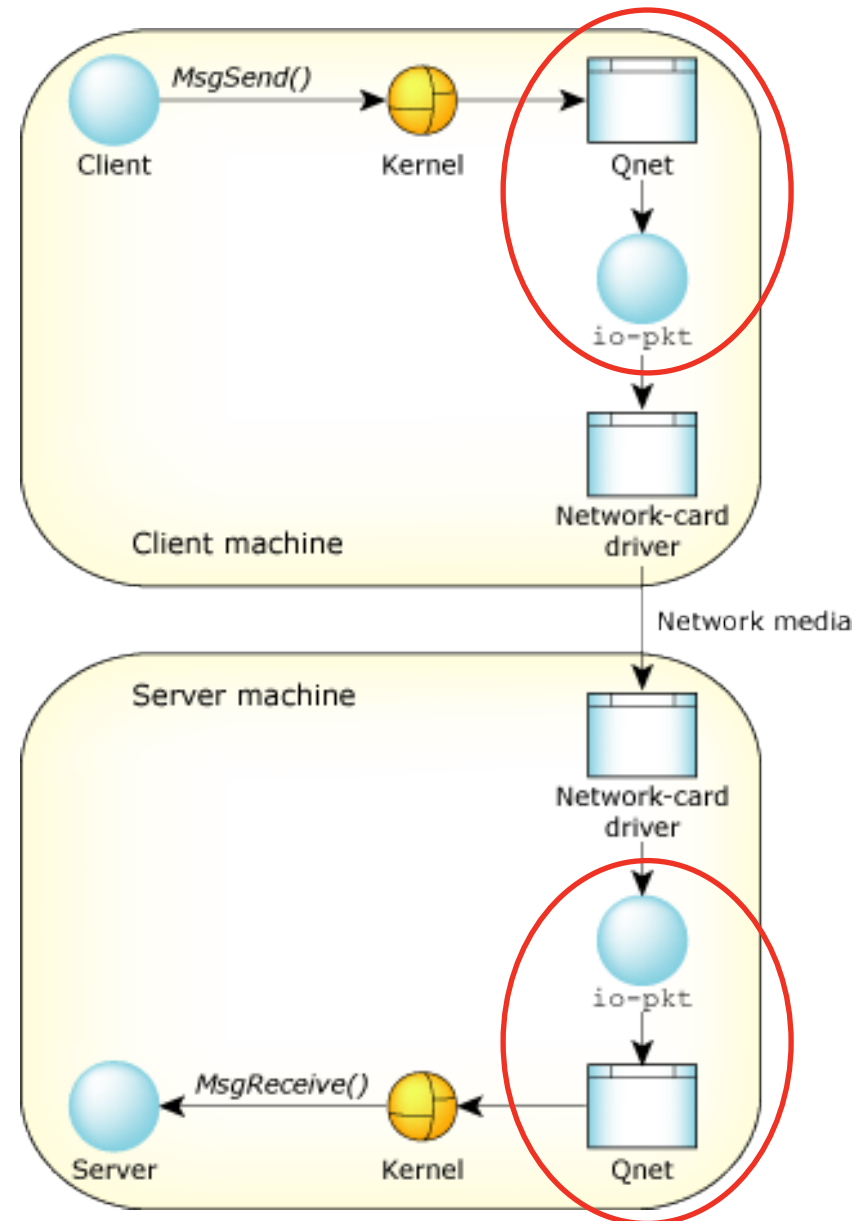
```
# mount -T io-pkt lsm-qnet.so
```



[1]

QNET – Enables Networked IPC

- The Qnet protocol extends interprocess communication (IPC) transparently over a network of microkernels.
- This is done by taking advantage of the Neutrino's message-passing paradigm.
 - Message passing is the central theme of QNX that manages a group of cooperating processes by routing messages.
 - The kernel puts the user data directly into (and out of) the network card's buffers - **there's no copying of the payload.**
 - **There are no context switches** as the packet travels from (and to) the kernel from the network card.
- Qnet's functionality is limited when applications create a shared-memory region.
 - That works best when the applications run on the the same machine/node.



References

Images and content used in this presentation can be found:

- [1] QNX Documentation - Get_Programming_with_the_QNX_Neutrino_RTOS.pdf
- [2] <http://www.freertos.org/implementation/a000004.html>
- [3] <http://embedindia.blogspot.com.au/2011/12/porting-of-micro-cos-ii-kernel-in-arm.html>
- [4] <https://www.micrium.com/hardware-accelerated-rtos-%C2%B5cos-iii-hw-rtos-and-the-r-in32m3/>
- [5] QNX Documentation - QNX_Neutrino_RTOS_System_Architecture.pdf
- [6] <http://technogems.blogspot.com.au/2011/08/easy-cross-platform-inter-process.html>
- [7] <https://blogs.msdn.microsoft.com/csliu/2009/12/20/parallel-computing-an-introduction/>

Acknowledgment

I would like to Thank and Acknowledge the support of: Dr. *Samuel Ippolito*, the Course Coordinator of corresponding “EEET2166 /1262 Real-time Systems Engineering & Design” course from RMIT Melbourne.

Side note: data-type size differences for 64 and 32-bit systems

- The QNX x86 TargetVM is a 64-bit system, whereas the ARMv7 targets are 32-Bit systems.
- Normally, this is no issue unless you are sending a data packet (struct) between different target architectures, such as in the case for Native Message Passing.

Data type	32-bit	64-bit
char	8	8
int	32	32
long	32	64 ^a
long long	64	64
off_t	signed 32	signed 64 ^b
paddr_t	unsigned 32	unsigned 64 ^b
ptrdiff_t	32	64
short	16	16
size_t, ssize_t	32	64
time_t	unsigned 32	signed 64 ^b
uintptr_t, intptr_t	32	64
void *	32	64

Many compound types have changed in size, including the following:

- [struct sigevent](#)
- [union sigval](#)
- [siginfo_t](#)
- struct iovec
- struct utimbuf
- [struct sigaction](#)
- struct _timer_info
- struct timeval
- and anything else that includes a base type whose size varies

Standard message header contains a signal that can cause a world of pain when setting up a data packet for message passing.

http://www.qnx.com/developers/docs/7.0.0/#com.qnx.doc.qnxsdpmigration/topic/data_types.html

Native QNX Message Passing Example

Both the Server and Client Threads need the following:

```
#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include <sys/dispatch.h>

#define ATTACH_POINT "myname"
#define BUF_SIZE 100 // size of reply message string
```

If QNET is running on the Targets, then on the **Client** side this can be a networked path such as:
"/net/Beagle01.net.intra/dev/name/local/myname"
However on the **Server** it should be written like this

```
typedef union { // Normally a union sigval
    union{
        _Uint32t sival_int;
        void *sival_ptr; // size 32 & 64-bit changes
    };
    _Uint32t dummy[4]; // dummy to reserve space
}_mysigval;
```

Custom union sigval that has the same size on both a 32 and 64-bit QNX target. Note the use of the dummy[4] variable.

```
typedef struct _Mypulse { // This replaced the standard
    _Uint16t type; // typedef struct _pulse msg_header_t;
    _Uint16t subtype;
    _Int8t code;
    _Uint8t zero[3]; // padding used in standard header
    _mysigval value;
    _Uint8t zero2[2]; // padding added to ensure the header has a standard size
    _Int32t scoid;
} msg_header_t;
```

Custom Pulse/Message header Struct. This replaces the standard _pulse struct so that code works on x86_64 and ARMv7 targets

```
typedef struct {
    struct _Mypulse hdr; // Custom header
    int ClientID; // our data (unique id from client that you program can use)
    int data; // our data <-- This is what we are here for
} my_data;
```

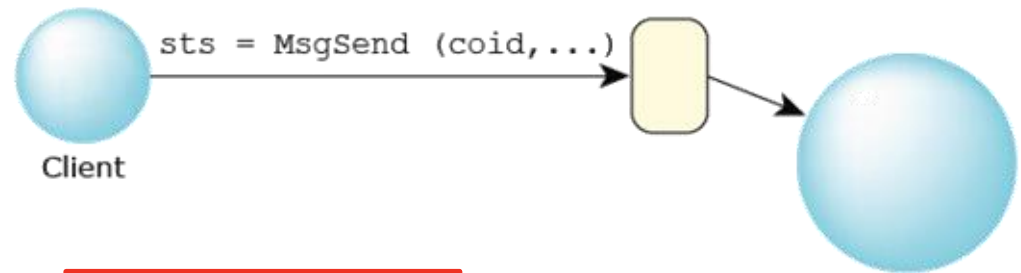
Struct header must be included first

```
typedef struct {
    struct _Mypulse hdr; // Custom header
    char buf[BUF_SIZE]; // Message to send back to send back to other thread
} my_reply;
```

Struct header must be included first

Server/Client Connections Using Communication Channels

- QNX also provides **faster** alternative function to `name_attach()` and `name_open()` to register a name (and create a channel) if you are **working on the same node**.
- To **receive** (Server) messages a thread creates a channel
 - `ChannelCreate()` (alternative to `name_attach()`)
 - `ChannelDestroy()` (alternative to `name_detach()`)
- To **send** (Client) messages a thread connects to a channel
 - `ConnectAttach()` (alternative to `name_open()`)
 - `ConnectDestroy()` (alternative to `name_close()`)
 - Can have multiple connections to a channel
- **A channel has three queues**
 - One for Receive blocked threads
 - One for Send blocked threads
 - One for Reply blocked threads



ConnCh-MsgPass-Server.c

ConnCh-MsgPass-Client.c

Make sure you include the correct PID of the running sever

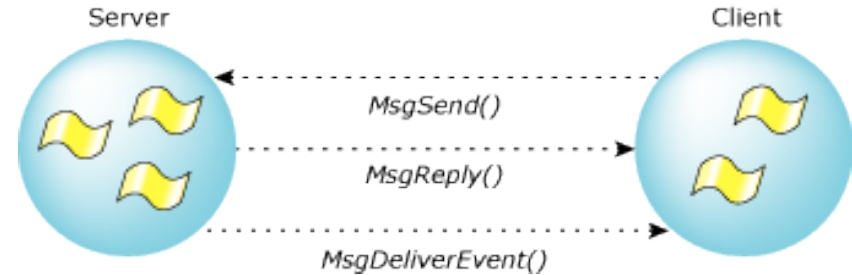
```

rcvid = MsgReceive (chid
// Processing happens
MsgReply (rcvid,...)
  
```

[1]

Multipart Messages, Priority

- The same `MsgReceive()`, `MsgSend()` and `MsgReply()` functions are use, however to create use: `ChannelCreate()`, `ConnectAttach()`
- The connection is closed with `ConnectDetach()` and `ChannelDestroy()` on the client and server, respectively.



- Multipart Messages versions**

- Use `MsgSendv()`, `MsgReceivev()`, `MsgReplyv()`
- These use an Input/Output Vector (IOV) to point the buffers containing each part of the message (avoids having to copy each message part into a single buffer before sending the message)

- Large Messages**

- Can send and receive in segments using `MsgRead()` and `MsgWrite()`

- The Send/Receive/Reply IPC primitives allow the construction of deadlock-free systems with the observation of only these simple rules:**

- Never have two threads send to each other
 - avoids standoff situations.
- Always arrange your threads in a hierarchy, with sends going up the tree.

